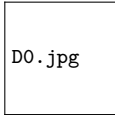


Lecture 15: Normalization (1NF to BCNF)

Eliminating Anomalies and Designing Good Schemas

Z2004: Database Management Systems



IIT Madras Zanzibar

Tuesday, 31 March 2026

0. Why Normalize? The Modern Tech Landscape

The Real-World Goal

Normalization is about creating a "Single Source of Truth." In a highly active system, if a piece of data exists in only one place, it can't contradict itself.

Where is strict Normalization used?

OLTP (Online Transaction Processing)

Systems with thousands of inserts/updates per second e.g., Banking systems, E-commerce checkouts, Airline reservations. We need writes to be lightning-fast and 100% safe from anomalies.

Do we ALWAYS Normalize?

No! In the modern data landscape, we sometimes intentionally *Denormalize* data to speed up reads.

Where is Denormalization used?

 **NoSQL & Document Stores:** (MongoDB, Firebase). Storing entire objects together prevents slow 'JOIN' operations.

 **OLAP (Data Warehouses):** (Snowflake, BigQuery). When analyzing massive amounts of historical data, we prefer "messy" flat tables because read-speed is more important than save-space.

The Golden Rule of Database Design

You must know how to perfectly Normalize a database before you are allowed to strategically Denormalize it!

1. The Problem with "Bad" Database Design

What is Normalization?

Normalization is a systematic, mathematical approach to decomposing tables to eliminate data redundancy and undesirable characteristics like **Insertion, Update, and Deletion Anomalies**.

The Goal: Ensure that every non-key attribute depends on *the key, the whole key, and nothing but the key* (so help me Codd!).

The Core Tool: FDs

- ▶ We use **Functional Dependencies (FDs)** from Lecture 12 to mathematically prove if a table design is structurally sound.

The Three Anomalies

When a database is not normalized, modifying it becomes dangerous and inefficient:

- ▶ **Update Anomaly:** Changing a piece of information requires updating multiple rows. If you miss one, the database becomes inconsistent.
- ▶ **Insertion Anomaly:** You cannot insert a new piece of data without having other unrelated data first.
- ▶ **Deletion Anomaly:** Deleting a row to remove one fact accidentally deletes a completely unrelated, important fact.

1. Visualizing the Anomalies

The "Messy" Table: Emp_Project

<u>EmpID</u>	EmpName	<u>ProjID</u>	ProjName	Hours
E01	Alice	P1	Alpha	10
E01	Alice	P2	Beta	20
E02	Bob	P1	Alpha	15
E03	Charlie	P3	Gamma	30

Update Anomaly

If Project Alpha is renamed to "Omega", we must update it for both Alice and Bob. If we forget Bob, the database is corrupted!

Insertion Anomaly

We want to add a new project "Delta". But we *cannot* save it until an employee is assigned to it (EmpID is part of the PK).

Deletion Anomaly

Charlie finishes Project Gamma and we delete his row. We have now completely lost the record that "Project Gamma" ever existed!

1b. The Normalization Roadmap & Terminology

Quick Terminology Refresher

Before we start decomposing, you must remember these two terms:

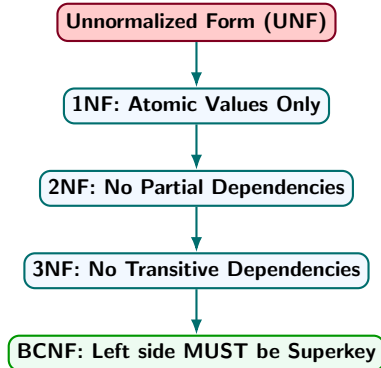
 **Prime Attribute:** An attribute that is part of a candidate key. (Usually underlined).

 **Non-Prime Attribute:** An attribute that is NOT part of any candidate key.

Example: In Student(SID, Name, Major)
SID is Prime.
Name and Major are Non-Prime.

The Normalization Pipeline

We step through normal forms strictly in order. You cannot reach 3NF without passing 1NF and 2NF first!



1c. Prime Attributes vs. Primary Keys

Clearing up a Common Misconception

Is a "Prime Attribute" just another word for the "Primary Key"? **No!**

The Definition:

A **Prime Attribute** is any attribute that is a member of *ANY* **Candidate Key**.

The Scenario: Vehicle Database

A car can be uniquely identified by its **VIN** (Vehicle Identification Number), OR by the combination of its **State** and **LicensePlate**.

Candidate Keys:

- ① { VIN }
- ② { State, LicensePlate }

Let's say the DBA chooses VIN to be the official **Primary Key**.

<u>VIN</u> (PK)	State	LicensePlate	Color
1ABC9...	TN	DBM-2026	Red

Which ones are Prime?

- ✓ VIN (It's in Candidate Key 1)
- ✓ State (It's in Candidate Key 2)
- ✓ LicensePlate (It's in Candidate Key 2)
- ✗ Color (This is our only **Non-Prime** attribute!)

Core Objectives of Normalization

The primary goals of normalization include:

- ▶ **Minimizing Data Redundancy:** Store each fact e.g., a customer's address in only one place.
- ▶ **Enhancing Data Integrity:** Enforce accurate, consistent data via keys and relationships.
- ▶ **Preventing Anomalies:**
 - ◇ **Insertion anomalies:** Can't add new data without also adding unrelated data.
 - ◇ **Update anomalies:** Same fact updated in some rows but not others.
 - ◇ **Deletion anomalies:** Deleting a row unintentionally removes useful information.

(Based on IBM documentation.)

The Normal Forms

Normalization is a **progressive process**: each normal form (NF) builds on the previous one and imposes **stricter rules** on the table design.

In this lecture, we'll focus on:

- ▶ **1NF**: atomic values (no repeating groups)
- ▶ **2NF**: no partial dependency on a composite key
- ▶ **3NF**: no transitive dependencies

Normalization Roadmap (Visual)

NORMALIZATION.png

2. First Normal Form (1NF)

The Rule for 1NF

A relation is in 1NF if and only if every attribute is **atomic** (indivisible).

Rule: Each column must contain only a single, atomic (indivisible) value, and there should be no repeating groups of columns.

Purpose: Eliminates repeating data within a single row and lays the foundation for a structured table.

- ▶ No arrays, lists, or comma-separated values in a single cell.
- ▶ No nested tables.

Example: `Customer(CustomerID, Name, Phones)` with `Phones = "555-0101, 555-0199"` violates 1NF; store one phone per row (or a separate Phone table).

2. First Normal Form (1NF) – Example

Violating 1NF

<u>Course</u>	Textbooks
CS101	<i>Intro to DB, SQL Guide</i>

Fixing it (Moving to 1NF)

To fix a 1NF violation, we split the comma-separated values into their own distinct rows.

<u>Course</u>	<u>Textbook</u>
CS101	Intro to DB
CS101	SQL Guide

✓ *Modern Note:* Some modern NoSQL databases (like MongoDB) intentionally break 1NF. But in strict relational theory, 1NF is mandatory.

3. Second Normal Form (2NF)

The Rule for 2NF

A relation is in 2NF if it is in 1NF AND it has **no partial dependencies**.

Rule: The table must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key. This rule applies to tables with a composite primary key (a key made of multiple columns).

Purpose: Removes partial dependencies, where a non-key attribute depends on only a portion of the composite key.

- ▶ A partial dependency occurs when a non-prime attribute depends on **only a part** of a composite primary key.
- ▶ If your Primary Key is only a single column, your table is *automatically* in 2NF!

Example: Enrollment(StudentID, CourseID, StudentName, CourseTitle) violates 2NF because StudentName depends only on StudentID and CourseTitle depends only on CourseID.

3. Second Normal Form (2NF) – Example

Violating 2NF

In Emp_Project, PK is (EmpID, ProjID).

- ▶ $\text{EmpID} \rightarrow \text{EmpName}$ (Partial!)
- ▶ $\text{ProjID} \rightarrow \text{ProjName}$ (Partial!)

Fixing it: Decomposition

We extract the partial dependencies into new tables, leaving the full dependency in the linking table.

Employee

<u>EmpID</u>	EmpName
--------------	---------

Project

<u>ProjID</u>	ProjName
---------------	----------

Works_On

<u>EmpID</u>	<u>ProjID</u>	Hours
--------------	---------------	-------

4. Third Normal Form (3NF)

The Rule for 3NF

A relation is in 3NF if it is in 2NF AND it has **no transitive dependencies**.

Rule: The table must be in 2NF, and there should be no transitive dependencies. This means non-key attributes should not depend on other non-key attributes.

Purpose: Ensures all attributes are directly dependent on the primary key, further reducing duplication and improving data integrity.

- ▶ A transitive dependency is when a non-prime attribute depends on **another non-prime attribute**.

Example: Employee(EmpID, EmpName, DeptID, DeptName) violates 3NF if $\text{EmpID} \rightarrow \text{DeptID}$ and $\text{DeptID} \rightarrow \text{DeptName}$ (so DeptName is transitively dependent on EmpID).

4. Third Normal Form (3NF) – Example

Violating 3NF

Student Table

<u>SID</u>	Name	Dept	DeptHead
------------	------	------	----------

- ▶ $SID \rightarrow Dept$ (Valid)
- ▶ $Dept \rightarrow DeptHead$ (Transitive! Non-prime determining non-prime).

Fixing it: Decomposition

We break off the transitive dependency into its own lookup table.

Student

<u>SID</u>	Name	Dept
------------	------	------

Department

<u>Dept</u>	DeptHead
-------------	----------

✓ Formal 3NF Definition:

For every non-trivial FD $X \rightarrow Y$, either:

- 1 X is a superkey, OR
- 2 Y is a prime attribute (part of a key).

5. Boyce-Codd Normal Form (BCNF)

The "Strict" 3NF (BCNF)

BCNF is stronger than 3NF. It removes the loophole allowing Y to be a prime attribute.

Formal BCNF Definition:

For every non-trivial FD $X \rightarrow Y$, X **MUST** be a superkey. No exceptions.

A 3NF table violating BCNF

Tutors Table

<u>StudentID</u>	<u>Subject</u>	Tutor
------------------	----------------	-------

- ▶ **FD1:** $(\text{StudentID}, \text{Subject}) \rightarrow \text{Tutor}$
- ▶ **FD2:** $\text{Tutor} \rightarrow \text{Subject}$ (Rule: Each tutor teaches only ONE subject).

Why does it fail BCNF?

Look at **FD2:** $\text{Tutor} \rightarrow \text{Subject}$.

- ▶ Is Tutor a superkey for the whole table? **No.** (A tutor has many students).
- ▶ Left side is NOT a superkey $\rightarrow$ Violates BCNF!

The BCNF Decomposition

We split based on the offending FD:

TutorSubject

<u>Tutor</u>	Subject
--------------	---------

StudentTutor

<u>StudentID</u>	<u>Tutor</u>
------------------	--------------

5. Boyce-Codd Normal Form (BCNF) – Another Example

BCNF Violation Example

Flights Table

<u>FlightNo</u>	<u>Date</u>	Gate
-----------------	-------------	------

Suppose the business rule is: *each gate handles only one flight per day.*

- ▶ **Key FD:** $(\text{FlightNo}, \text{Date}) \rightarrow \text{Gate}$
- ▶ **Extra FD:** $(\text{Gate}, \text{Date}) \rightarrow \text{FlightNo}$

Problem: $(\text{Gate}, \text{Date})$ is **not** a superkey, but it determines FlightNo .

BCNF Decomposition

Decompose on the offending FD $(\text{Gate}, \text{Date}) \rightarrow \text{FlightNo}$.

GateAssignment

<u>Gate</u>	<u>Date</u>	FlightNo
-------------	-------------	----------

FlightSchedule

<u>FlightNo</u>	<u>Date</u>	Gate
-----------------	-------------	------

Now, in each table, every non-trivial FD has a superkey on the left.

5b. 3NF vs. BCNF: The "Gold Standard"

"If I am in 3NF, aren't I mostly fine?"

In the real world, the answer is usually **Yes**. Most tables that reach 3NF are already naturally in BCNF. However, 3NF leaves open a very specific mathematical loophole.

The 3NF Loophole

Recall that 3NF allows $X \rightarrow Y$ if Y is a **Prime Attribute**.

- ❗ This means a non-key attribute is allowed to determine *part* of a Candidate Key!
- ❗ This rare scenario only happens if a table has **overlapping composite candidate keys** (like our Tutor/Subject example).
- ❗ **The result:** Slight data redundancy and update anomalies can still sneak in.

The BCNF Guarantee

BCNF strictly states: For $X \rightarrow Y$, X **MUST** be a superkey. No exceptions for Y .

BCNF closes the 3NF loophole entirely.

It guarantees absolutely **zero redundancy** caused by functional dependencies.

The result: This strictness makes BCNF the true mathematical "Gold Standard" of relational database design.

🏆 The Bottom Line

If a table is in BCNF, it is automatically in 3NF. But if it is in 3NF, you must still check for overlapping candidate keys to ensure it reaches BCNF!

Class Activity: Spot the Violation

The Scenario

A local clinic stores patient appointments in a single table:

Appointment

ApptID	PatID	PatName	DocID	DocName	Date
--------	-------	---------	-------	---------	------

Functional Dependencies (FDs):

- 1 $\text{ApptID} \rightarrow \text{PatID}, \text{PatName}, \text{DocID}, \text{DocName}, \text{Date}$
- 2 $\text{PatID} \rightarrow \text{PatName}$
- 3 $\text{DocID} \rightarrow \text{DocName}$



Instructions

Take 5 minutes to solve the two tasks on the right.

Task 1: Identify the Anomaly

What specific anomaly occurs if Doctor "Dr. Smith" ($\text{DocID} = \text{D1}$) quits, and we delete all of his upcoming appointments from this table? Why?

Task 2: Normalize the Schema

1. What Normal Form is this table currently violating? (1NF, 2NF, 3NF, or BCNF?)
2. Decompose this single table into a set of visual tables that fully satisfy BCNF. Be sure to underline the Primary Keys.

Class Activity: Solutions

Task 1 Solution

Identify the Anomaly:

This is a **Deletion Anomaly**.

Why? If we delete the appointment rows to remove Dr. Smith's schedule, we simultaneously delete the PatName of the scheduled patients. We lose the record of those patients' names entirely from the clinic's database!

Task 2 Solution (Part 1)

What is the violation?

The table violates **3NF**. The Primary Key is ApptID. Both $\text{PatID} \rightarrow \text{PatName}$ and $\text{DocID} \rightarrow \text{DocName}$ are **transitive dependencies** (non-prime determining non-prime).

Task 2 Solution (Part 2)

The BCNF Decomposition:

We split out the transitive dependencies into their own tables. All left-hand sides of FDs are now superkeys!

Patient

<u>PatID</u>	PatName
--------------	---------

Doctor

<u>DocID</u>	DocName
--------------	---------

Appointment

<u>ApptID</u>	PatID	DocID	Date
---------------	-------	-------	------

Lecture Summary & Next Steps

Key Takeaways

- ✓ **Anomalies:** Poor design leads to Insertion, Update, and Deletion errors.
- ✓ **1NF:** Atomic values only. No arrays or lists.
- ✓ **2NF:** No Partial Dependencies. Non-prime attributes must depend on the *entire* primary key.
- ✓ **3NF:** No Transitive Dependencies. Non-prime attributes cannot depend on other non-prime attributes.
- ✓ **BCNF:** For every $X \rightarrow Y$, X must be a superkey. The strictest standard form.

Next Lecture (L16)

Decomposition & 4NF

(Thursday, 2 April 2026)

- ➔ Multivalued Dependencies (MVDs).
- ➔ Fourth Normal Form (4NF).
- ➔ Lossless-Join Decomposition Testing.
- ➔ Dependency Preservation Testing.

Reading Assignment



Read **Silberschatz, Ch. 8**

(Decomposition Algorithms) to prepare for the heavy math on Thursday!